

六个值得关注的 Claude Skills

陈成推文盘点全解读 · 2 个深挖动手 + 1 个架构精读 + 3 个盘点

来源: x.com/chenchengpro/status/2095461259949052264 (2026-09) | 拾遗任务 d7ab71a0 | 2026-10-03 | reminder 学习仓库

 reverse-skill 39.3k★

 dev-pair

 claude-api 官方

 gen-pptx

 implement-spec 主学

 avoid-ai-writing 主学

Q1 这条推文到底盘了哪 6 个 skill?

Q2 skill 不就是提示词吗, 凭什么值得"关注"?

Q3 implement-spec: 一个 spec 怎么变成一屋子并行施工的 agent?

Q4 to-spec 模板里为什么"禁止写文件路径"?

Q5 avoid-ai-writing 怎么"闻"出 AI 味?

Q6 为什么作者反而劝你"别拿它定罪"?

Q7 reverse-skill: 39k★ 的技能路由器怎么工作?

Q8 授权硬门: 为什么"点名目标≠授权"?

Q9 dev-pair: 为什么让另一个模型来挑刺?

Q10 盘点: gen-pptx 与官方 claude-api

Q11 主实验: implement-spec 微型演练场 (真实跑通)

Q12 实验二: AI 味检测器实测 44→0 (真实跑通)

Q13 能力清单: 我能拿它们干什么?

Q1

这条推文到底盘了哪 6 个 skill?

陈成 (@chenchengpro) 用一条推文列出了他最近认为**值得关注的 6 个 Claude skills**。全部核实过：6 个都是真实存在、可克隆的开源仓库：

#	Skill	仓库	热度	一句话定位	本篇处理
1	reverse-skill	zhaoxuya520/reverse-skill	39.3k★	逆向/渗透安全技能 路由器包 ：先路由、再授权、再干活	架构精读
2	dev-pair	justinjohnson25600/hermes_skills/dev-pair	—	让 另一个 LLM 当"第二双眼睛"挑刺，带硬预算	盘点
3	claude-api	anthropics/skills (官方)	179k★	官方 API 参考：防模型 凭过 时记忆写错代码	盘点
4	gen-pptx	JimLiu/baoyu-design	4.2k★	宝玉系：HTML 设计稿 → PPTX 成品管线	盘点
5	implement-spec	mattpocock/skills (engineering)	—	spec → 工单任务图 → 并行施工 → 合并 (TypeScript 圈名人的工程方法论)	主学+实验
6	avoid-ai-writing	conorbronsdon/avoid-ai-writing	4.8k★	74 类 AI 味模式库 + 零依赖确定性检测器	主学+实验

为什么这样分工？今天流水线已连学多个 skill 类项目（中医 skill、生图 skill、agent-reach.....），**全部铺开会重复占坑**。所以挑**方法论增量最大**的两个（implement-spec 讲"怎么组织施工"、avoid-ai-writing 讲"怎么质检文字"）深挖动手，reverse-skill 做架构精读（它的"路由器"形态最独特），其余三个盘点认识即可。

Q2

skill 不就是提示词吗，凭什么值得"关注"？

对，SKILL.md 本质是一段按需加载的提示词（我们知识库已有 [[AgentSkills技能包]]）。但这 6 个展示了 skill 的**三种进阶形态**，已经远超"一段话教模型做事"：

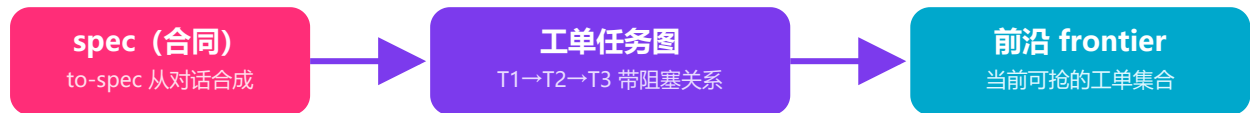
形态	代表	比"提示词"多出来的东西
编排器	implement-spec	指挥子代理并行干活：任务图、分支策略、合并节奏——skill 是 施工总指挥的作业规程
检测器+提示词	avoid-ai-writing	提示词之外还带 可执行代码 ：零依赖 Node 检测器、CI 门槛、引文保护校验
路由器+硬门	reverse-skill	45 个子技能 + 44 条结构化路由规则 + 授权门 + 经验回流——skill 是 分诊台不是医生

一个诚实的提醒：官方 claude-api skill 的价值恰恰是"反向"的——模型对 API 的记忆会过期，**与其信训练记忆，不如信随包分发的权威参考**。这也是 skill 生态存在的根本理由之一：把"会过期的知识"从模型权重里搬到仓库里，随 git 更新。

Q3

implement-spec: 一个 spec 怎么变成一屋子并行施工的 agent?

这是 Matt Pocock (Total TypeScript 作者) "Skills for Real Engineers" 系列。核心洞见: **工单 (ticket) 不是步骤清单, 是带阻塞关系的任务图**——所以任何时刻都存在一个"没人挡路的工单"集合, 术语叫 **frontier (前沿)**, 可以同时派给多个施工子代理。



前沿随完工推进: 开工时 $\text{frontier} = \{T1\}$

T1 完工瞬间 $\rightarrow \text{frontier} = \{T2\}$, T2 的施工代理立刻上场

每个施工代理在自己的 git worktree 分支干活 $\rightarrow$ 完工 merge 回 integration 分支 $\rightarrow$ 收尾统一 code-review

- **通信靠"指针"不靠转述**: 子代理之间传递的是"去看 spec 第 X 节 / 去看某 commit", 不重复拷贝信息。
- **每个施工代理**先确认自己的 worktree 基于 integration 分支, 用 **tdd** 技能建工单, 完工前先合并主干最新代码再交卷。
- **收尾**: 全部工单闭合后统一跑 code-review, 所有问题由一个施工代理一次修完。

和我们知识库的 [[GitWorktree并行隔离]]、[[规格书先行SpecDriven]] 完全同源——这个 skill 把两条已有概念**组装成了完整施工流水线**。

Q4

to-spec 模板里为什么"禁止写文件路径"?

上游配套的 to-spec 技能规定了 spec 模板: 问题陈述 / 方案 / **超长的用户故事清单** / 实现决策 / 测试决策 / 范围外。最妙的一条规矩:

"不要写具体文件路径和代码片段——它们很快就会过时。" 例外: 如果原型产出了比文字更精确的决策载体 (状态机、schema、类型形状), 可以内联, 且只留"决策密集"的部分。

道理和 [[规格书先行SpecDriven]] 一脉相承: spec 要写的是 **会长期成立的"决策"**, 不是会天天变的"位置"。另外它引入了 **seam (接缝)** 概念——测试从哪里伸进去摸到代码。规矩是: **优先复用现有接缝、能用高层接缝就不开新的、理想接缝数是一**。接缝越少, 测试越像"从窗外看房子"而不是"撬开每面墙检查电线"。

Q5 avoid-ai-writing 怎么"闻"出 AI 味?

这个 skill (v3.36, MIT, 4.8k★) 把 LLM 写作的高频套路整理成 **74 个模式类目**, 并按严重程度分三级:

级别	名字	典型例子	处理建议
P0	可信度杀手	"截至我最后一次更新"、聊天客套"I hope this helps!"、无出处的"专家认为"	立刻修
P1	明显 AI 味	高频词 delve/robust/leverage; "In the rapidly evolving world of..."开场; Moreover/Furthermore 过渡; 排比三连	发布前修
P2	风格抛光	em dash 频率 (>1/千词)、段落等长、The future looks bright 式结尾	有空再修

光有目录不算本事——仓库还带一个 **零依赖 (只用 Node 标准库) 的确定性检测器**, 把能写成规则的子集变成可执行打分:



```
# 一条命令体检一篇文章 (零安装, Node≥18)
node repo/bin/avoid-ai-writing.js draft.txt
# 还能当 pre-commit / GitHub Action 门槛:
avoid-ai-writing-gate --glob "**/*.md" --context technical # 默认阈值 6 处/文件
```

阈值 6 不是拍脑袋: 在 376 篇人写文档的对照语料上, 阈值 0 会冤枉 **31.4%** 的人写文档, 阈值 6 只冤枉 **1.9%**——**默认值是量出来的**。

Q6

为什么作者反而劝你"别拿它定罪"?

SKILL.md 开头用很长一段"泼自己冷水", 这在 skill 里很少见, 也是最值得学的部分:

- 独立审计发现商用 AI 检测器对**非母语写作者**误报率超 60% (Stanford, *Patterns* 2023) ; 开源检测器整体错判率超 70%; 对抗性改写能让所有检测方法准确率掉 ~88%。
- 赶稿的人、二语写作者、刻意压缩词汇的技术文体, 会"天然长出"同样的 AI 形状。

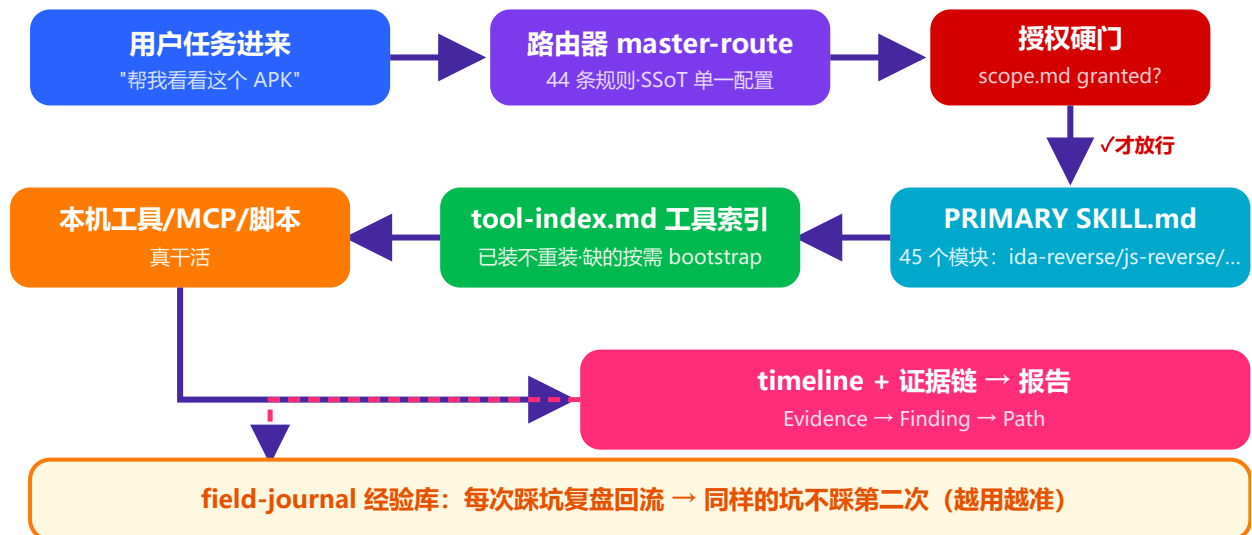
"**信号, 不是证据 (signals, not proof)**" ——可以用来清理自己的文章, **不能单独当作学术诚信、招聘、署名归因的判据。**

它还有一套"改写者自律条款": 改写时 **不许编造原文没有的细节/立场/经历** (否则等于给文本换了另一副 AI 指纹)、引用和代码是保护区不动、检测到问题不等于获准改写 (模式要经语境豁免过滤才算"发现")。这和我们知识库的 [[证据优先质检ProofOverClaims]]、[[事实与判断分离]] 完全同一个价值观。

Q7

reverse-skill: 39k★ 的技能路由器怎么工作?

当 AI 面对一道 APK、ELF 二进制、前端 JS 加密或 CTF 题时, 问题是 "**不知道该上 jadx、Frida 还是 IDA**" ——技能太多, 选型靠抓阄。reverse-skill 的答案: 先分诊, 再干活。



三个设计点值得抄走:

- **配置是唯一事实源 (SSoT)**: 路由表只在 `skills/config/routing.json` 一处, Markdown 表只是给人看的投影; 175 条回归用例 + Windows/Ubuntu 双平台 CI 守着它。
- **分层按需读取**: 平时只看 MASTER-ROUTING 快路径, PRIMARY 有歧义才查全表 ——技能包越大, "不预加载"越值钱 ([[分层按需加载]])。
- **工具自举**: tool-index.md 记录本机哪些工具可用, 已装的不重复装, 缺的披露影响、获批后才 bootstrap ——换机器不散架。

Q8

授权硬门：为什么"点名目标≠授权"？

安全类技能包天然危险：AI 拿到渗透方法论，万一"手滑"对没授权的目标动手呢？reverse-skill 的 RULES_zh.md 写得极其清醒：

- "读仓库文件 ≠ 授权执行仓库脚本"——只读任务（审查/摘要）必须保持只读；
- 任何本机副作用（写文件、下载、起服务、联网）之前，先列出准确命令清单，明确同意后才动手；
- 目标授权是独立的硬门：点名目标不等于 granted，--force 永远绕不过 scope 检查；
- 想"跳过已获批步骤"或"只停在确认回复磨洋工"？仓库甚至准备了"借口反驳表"（agent-obedience-engineering.md）两头都堵。

这和知识库 [[ClaudeCodeHooks与fail-open]] 恰好构成一对镜像：遥测观测挂了要放行（fail-open，别连累业务）；安全授权没拿到必须拦死（fail-closed）。什么时候该 fail-open、什么时候该 fail-closed，是 Agent 工程的一对基本判断。

Q9

dev-pair：为什么让另一个模型来挑刺？

dev-pair（hermes_skills 仓库）的逻辑：干活的模型自己查自己，容易"自己骗自己"；换一个不同厂家的 LLM 来做纯监督式 review——只挑方向、找 bug、提刁钻问题，不代写、不重做一遍。更有意思的是它的"防滥用三件套"：

机制	作用	Agent 能绕过吗？
daily_cap 每日预算	付费调用数的硬上限，进程直接拒绝	不能（真正的锁）
append-only 台账	每次调用先记账再打钱，可审计	不能（但只记录）
--requested-by 署名	声明"谁要求的"	能（只是自我声明）

skill 还规定：agent 觉得该请第二意见时，只许用一句话提议然后闭嘴等用户点头，不许"为了周到"自己偷偷跑。台账写不进去时整个调用失败关闭（fail-closed）——又是 Q8 那对镜像。



盘点：gen-pptx 与官方 claude-api

gen-pptx (JimLiu/baoyu-design)

宝玉 (baoyu) 系 skill 家族的生成管线，藏在 baoyu-design 仓库的 [skills/baoyu-design/agents/gen-pptx](#)。思路：让 LLM 写 **HTML 设计稿**（它最擅长的格式），再用浏览器捕获管线（截图 / 可编辑元素捕获双路）转成真正的 PPTX。和我们做 PDF 的“HTML → Edge 无头打印”是同一个哲学：HTML 是 LLM 的母语，排版引擎负责最后一步。baoyu-design 整仓 4.2k★，主打“本地版 Claude Design”。

claude-api (anthropics/skills 官方)

官方技能仓库 (179k★) 里的 API 参考 skill。要解决的是 **模型记忆过期** 问题：模型凭训练记忆写 API 调用，型号、参数、定价可能全是旧的。解法是把权威参考随 skill 分发——“会过期的知识放仓库，随 git 更新，不放进权重”。本会话里它已经作为内置技能在场，算是最“零距离”的一个。

Q11

主实验：implement-spec 微型演练场（已真实跑通）

目标：用最小代码体会"spec → 工单任务图 → 分支施工 → 合并验收"全流程。实验材料在 [F:\reminder\six_skills\exercise\implement-spec-lab\](#)（独立 git 仓库）。

Step 1 | 写 spec + 工单任务图 (to-spec 产出物)

目的：把"词频统计小工具"固化成合同和带阻塞关系的工单图

```
cat > spec.md      # 问题/方案/用户故事/实现决策/测试决策/范围外
cat > tickets.md    # T1 tokenize (无阻塞) → T2 report (依赖T1) → T3 cli (依赖T1+T2)
```

✅ 预期：git commit 成功，tickets.md 里 T1 标注"frontier 初始成员"

Step 2 | T1 施工：开分支实现 tokenize

目的：模拟"施工子代理在 own worktree/branch 干活"

```
git checkout -b t1-tokenize
# src/tokenize.py: re.findall(r"[a-z]+", text.lower())
python tests/test_t1.py
```

✅ 预期输出：T1 tests: PASS ("The cat, the dog!" → ["the","cat","the","dog"])

Step 3 | T1 完工 → 合并 integration 分支 → T2 上场

目的：体会"前沿推进"——T1 一合并，T2 立刻从"被阻塞"变"可施工"

```
git checkout main && git merge t1-tokenize
git checkout -b t2-report    # Counter + 按(-次数, 字母序)排序
python tests/test_t2.py     # PASS: [("b",2),("a",1)], 同频按字母序
git checkout main && git merge t2-report
```

✅ 预期：T2 tests PASS，两次 merge 无冲突（接口契约在 spec 里定死了）

Step 4 | T3 收尾：命令行入口 + 端到端验收

目的：三件套合体，跑"全量测试 + 真实数据"当最终 Gate

```
python topwords.py sample.txt -n 3
# sample.txt = "The quick brown fox jumps over the lazy dog. The dog barks; the fox runs.
The end."
```

```
5 the
2 dog
2 fox
```

✅ 预期：输出如上 + T1/T2 测试重跑全 PASS + git log 显示 T1/T2/T3 三个分支先后合入 main (共 6 个 commit)

Step 5 | 对照真实 skill 的差异 (诚实记录)

目的：知道简化版少了什么

真 skill 用**并行子代理 + worktree** (我们顺序手工跑)、用**issue tracker 关单** (我们用 merge 代替)、收尾跑**code-review 技能** (我们用全量测试代替)。骨架——任务图、前沿、integration 分支、TDD 施工——全部保留。

✅ 坑记录：Windows Python 3.14 没装 pytest → 改用 stdlib 断言脚本，反而更"零依赖"

Q12

实验二：AI 味检测器实测 44 → 0（已真实跑通）

材料在 F:\reminder\six_skills\exercise\（上游 repo 已克隆，commit bdeb726）。

Step 1 | 故意写一段"AI 味爆炸"的英文，跑检测器

```
printf 'In the rapidly evolving world of software development, leveraging robust tools
is essential. Let me delve into why this matters. ...' > ai_sample.txt
node repo/bin/avoid-ai-writing.js ai_sample.txt > detector_ai_sample.json
```

✅ 实测：得分 44 (Moderate AI signals)，9 处命中——robust、delve、leveraging、Moreover、"I hope this helps"..... 每条还带替换建议（delve → explore, dig into）

Step 2 | 按 SKILL.md 的规则改写（删高频词、砍空开场、去掉客套尾巴）

```
node repo/bin/avoid-ai-writing.js ai_sample_rewrite.txt > detector_ai_rewrite.json
```

✅ 实测：得分 0 (Clean)，0 处命中。同法测人味对照段 human_sample.txt 也是 0 分——检测器不冤枉好人。

同一套规则还能反过来用：本仓库的中文学习指南也可以在定稿前过一遍这个检测器的**结构性规则**（排比三连、空洞开场、significance inflation——"意义膨胀"是跨语言的）。



能力清单：我能拿它们干什么？

能力	具体用法（结合本仓库）
📋 任务图拆工单	下次给大型学习项目排期时，先写 spec + 带阻塞的 tickets，明确哪些可以并行（哪怕执行者是自己）
🔥 给文档"验 AI 味"	node bin/avoid-ai-writing.js 指南.html ——产出 PDF 前自检一遍，重点抓"意义膨胀"和空洞开场
🚦 CI 写作门槛	avoid-ai-writing-gate 挂 pre-commit，超过阈值 6 处就不让提交英文文档
🗺️ 路由器模式	本仓库 400+ 子项目越来越多，可以借"路由表 + SSoT 配置 + 条件读取"组织 vault 的检索
🔒 授权硬门	任何会动真环境的实验，先写 scope（目标/边界/禁止项），granted 才跑——照抄 reverse-skill 的 RULES 纪律
💡 跨模型第二意见	重要报告可以请另一个模型做纯挑刺式 review（记住 daily_cap 思想：预算先行，先记账后花钱）

一句话带走：这 6 个 skill 合起来在回答同一个问题——**当 AI 的能力够了之后，"组织和纪律"就成了新的瓶颈**：施工要任务图，写作要模式库，安全要授权门，花钱要硬预算。